CHAPTER 38 · DEVSECOPS LAB BOOK

Advanced Supply Chain Security Deep Dive

Securing the entire pipeline — from source to signed, verifiable artifact.



SBOM



SLSA



Signing



Dependencies



Containers

What We'll Cover

Five pillars of a hardened, verifiable software supply chain

Software Bill of Materials

Generate, validate, and sign a complete inventory of every component in your build.

01

SLSA Framework — Levels 1–4

A maturity model for build integrity, from documented process to hermetic, fully verified builds.

02

Artifact Signing & Verification

Cryptographically sign images and attestations with cosign; enforce signatures at deploy time.

03

Dependency Management

Scan, auto-remediate, and lock third-party dependencies across ecosystems.

04

Container Image Security

Scan, sign, attest, and enforce policy on every image before it reaches production.

05

Software Bill of Materials (SBOM)

A complete, signed inventory of everything that goes into your build

GENERATE · VALIDATE · SIGN

```
# Generate SBOM with Syft
syft alpine:latest \
  -o spdx-json > sbom.json

# Validate SBOM
sbom-utility validate \
  -f sbom.json

# Sign SBOM with cosign
cosign sign-blob \
  --key cosign.key \
  sbom.json > sbom.json.sig
```

SBOM Contents

- Package name & version
- License information
- Dependency tree
- Known vulnerabilities
- Supplier information
- Creation timestamp

SLSA Framework — Levels 1 to 4

A maturity model for supply chain integrity — each level raises the bar

LEVEL 1

Provenance Only

- Build process documented
- Some version control
- No verification

LEVEL 2

Basic Integrity

- Hosted build platform
- Access controls
- Build logs retained
- Provenance generated

LEVEL 3

Advanced Integrity

- Security hardened build
- Secrets management
- Code review required
- Reproducible builds
- Artifact signing

LEVEL 4

Complete Supply Chain

- Multiple independent reviews
- Hermetic builds (no external inputs)
- Complete transparency
- Verified dependencies
- Secure distribution

Artifact Signing & Verification

cosign secures the chain from keypair to Kubernetes admission

Generate Keys

```
cosign generate-key-pair
```

Sign Image

```
cosign sign --key cosign.key
harbor.devsecops.com/app:v1.0
```

Attest Build

```
cosign attest --key cosign.key
harbor.devsecops.com/app:v1.0
```

Verify Signature

```
cosign verify --key cosign.pub
harbor.devsecops.com/app:v1.0
```

Enforce in Kubernetes: Gatekeeper or Kyverno admission policies reject any Pod whose image lacks a valid cosign signature — turning verification into a hard deployment gate.

Dependency Management

Scan for known risk, auto-remediate, then lock what you trust

Scan Dependencies

```
npm audit --json > \
  npm-audit.json

pip-audit --desc > \
  pip-audit.txt

go list -json ./... | \
  nancy sleuth
```

Auto-Remediate

```
dependabot check \
  --auto-update

renovate \
  --runOnboarding
```

Lock Versions

```
npm ci \
  --prefer-offline

pip install -r \
  requirements.lock

go mod download
```

Container Image Security

Every image is scanned, signed, attested, and policy-enforced before it runs

Scan for Vulnerabilities

```
trivy image --severity HIGH,CRITICAL harbor.devsecops.com/app:latest
```

Sign the Image

```
cosign sign --key key.pem harbor.devsecops.com/app:latest
```

Verify the Signature

```
cosign verify --key key.pub harbor.devsecops.com/app:latest
```

Create Attestation

```
cosign attest --predicate provenance.json --key key.pem
harbor.devsecops.com/app:latest
```

Enforce with Kyverno

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: verify-signatures
spec:
  validationFailureAction: enforce
  rules:
  - name: check-signature
    match:
      resources:
        kinds: [Pod]
    verifyImages:
    - imageReferences:
      - 'harbor.devsecops.com/*'
      attestations:
      - name: provenance
        predicateType: \
          https://slsa.dev/provenance/v0.2
```

Key Takeaways

Trust nothing by default — verify every link in the chain

Inventory everything

An SBOM turns “what's in this build?” into an answerable, signable question.

Climb the SLSA levels

Move deliberately from documented builds toward hermetic, fully verified pipelines.

Sign and verify, always

cosign attestations plus admission policies close the gap between build and deploy.

Automate dependency hygiene

Continuous scanning, auto-remediation, and lockfiles keep drift from becoming risk.